Marco de Desarrollo

Sesión 10

Servicios Spring: Relaciones de Tabla

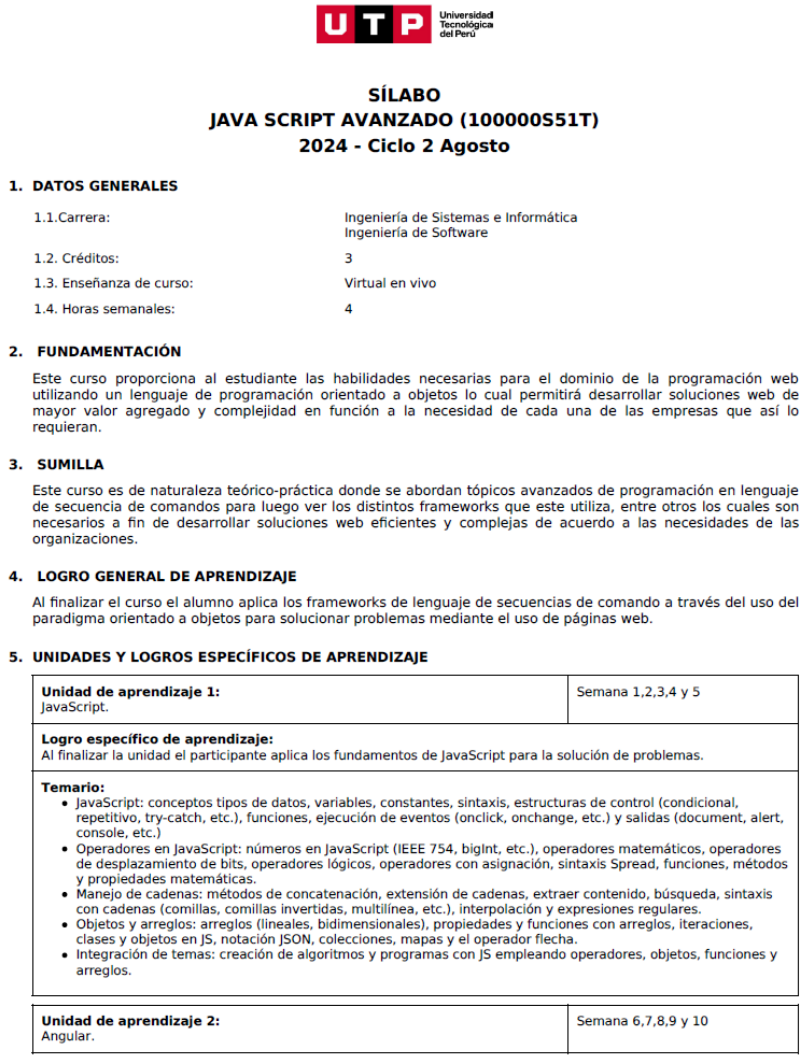
Mg. Alberto Moreno



Revisamos el Sílabo

Dentro de este documento encontraremos información sobre el curso, incluyendo:

- Logro del curso.
- Cronograma de actividades.
- Sistema de evaluación.
- Bibliografía.



SÍLABO
JAVA SCRIPT AVANZADO (100000S51T)
2024 - Ciclo 2 Agosto

1. DATOS GENERALES

1.1.Carrera:	Ingeniería de Sistemas e Informática Ingeniería de Software
1.2. Créditos:	3
1.3. Enseñanza de curso:	Virtual en vivo
1.4. Horas semanales:	4

2. FUNDAMENTACIÓN

Este curso proporciona al estudiante las habilidades necesarias para el dominio de la programación web utilizando un lenguaje de programación orientado a objetos lo cual permitirá desarrollar soluciones web de mayor valor agregado y complejidad en función a la necesidad de cada una de las empresas que así lo requieran.

3. SUMILLA

Este curso es de naturaleza teórico-práctica donde se abordan tópicos avanzados de programación en lenguaje de secuencia de comandos para luego ver los distintos frameworks que este utiliza, entre otros los cuales son necesarios a fin de desarrollar soluciones web eficientes y complejas de acuerdo a las necesidades de las organizaciones.

4. LOGRO GENERAL DE APRENDIZAJE

Al finalizar el curso el alumno aplica los frameworks de lenguaje de secuencias de comando a través del uso del paradigma orientado a objetos para solucionar problemas mediante el uso de páginas web.

5. UNIDADES Y LOGROS ESPECÍFICOS DE APRENDIZAJE

Unidad de aprendizaje 1: JavaScript.	Semana 1,2,3,4 y 5
Logro específico de aprendizaje: Al finalizar la unidad el participante aplica los fundamentos de JavaScript para la solución de problemas.	
Temario: • JavaScript: conceptos tipos de datos, variables, constantes, sintaxis, estructuras de control (condicional, repetitivo, try-catch, etc.), funciones, ejecución de eventos (onclick, onchange, etc.) y salidas (document, alert, console, etc.) • Operadores en JavaScript: números en JavaScript (IEEE 754, bigInt, etc.), operadores matemáticos, operadores de desplazamiento de bits, operadores lógicos, operadores con asignación, sintaxis Spread, funciones, métodos y propiedades matemáticas. • Manejo de cadenas: métodos de concatenación, extensión de cadenas, extraer contenido, búsqueda, sintaxis con cadenas (comillas, comillas invertidas, multilínea, etc.), interpolación y expresiones regulares. • Objetos y arreglos: arreglos (lineales, bidimensionales), propiedades y funciones con arreglos, iteraciones, clases y objetos en JS, notación JSON, colecciones, mapas y el operador flecha. • Integración de temas: creación de algoritmos y programas con JS empleando operadores, objetos, funciones y arreglos.	

Unidad de aprendizaje 2: Angular.	Semana 6,7,8,9 y 10

Logro de aprendizaje

Al finalizar la sesión, el participante aplica los tipos de relaciones de tabla con Jpa para obtenerconsultas mediante ejercicios para la solución de problemas.



Imagen obtenida de:
https://img.freepik.com/free-vector/clever-man-student-standing-books-stack-with-flag-self-learning-personal-improvement-knowledge-obtaining-educational-achievement_335657-3461.jpg

Inicio

¿Sabías que…?

Las **relaciones en JPA** son fundamentales para modelar correctamente el dominio de una aplicación y representar cómo interactúan las entidades entre sí.



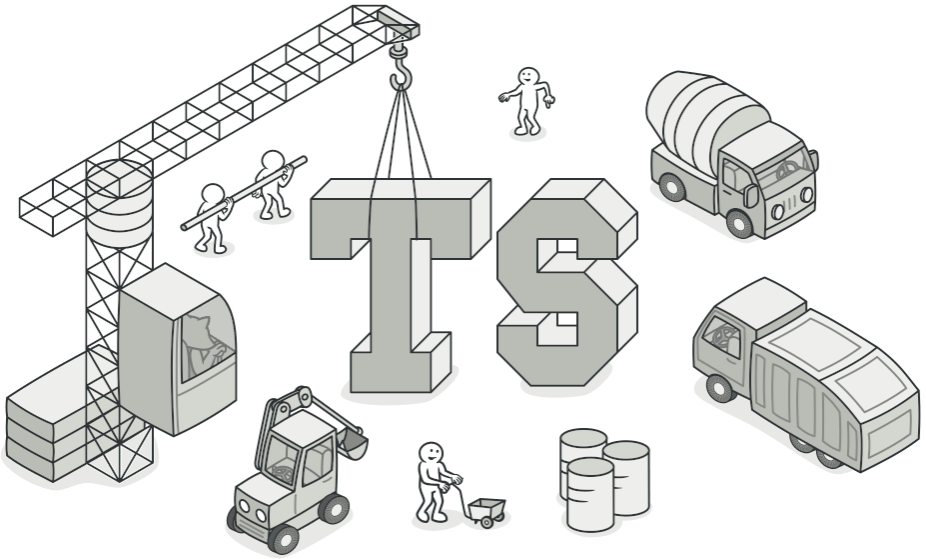
Imagen obtenida de:
https://img.freepik.com/free-vector/clever-man-student-standing-books-stack-with-flag-self-learning-personal-improvement-knowledge-obtaining-educational-achievement_335657-3461.jpg

Utilidad

Reflejan el modelo real de negocio

- Permiten mapear las **relaciones reales del mundo** (por ejemplo, un cliente tiene muchos pedidos, un producto pertenece a una categoría, etc.). Esto hace que tu código sea más claro y mantenible.

.



contenido

1. Introducción a JavaScript
2. JavaScript tipos de datos, Constantes y Estructuras de Control
3. Funciones y ejecución de Eventos
4. Salidas

Transformación

1 Introducción a JavaScript

Tipos de relaciones

Tipos de Relaciones en JPA

Relación	Significado
`@OneToOne`	Uno a uno
`@OneToMany`	Uno a muchos
`@ManyToOne`	Muchos a uno
`@ManyToMany`	Muchos a muchos

1. 🔁 `@ManyToOne` y `@OneToMany`

Ejemplo: Una **categoría** tiene muchos **productos**, pero cada **producto** pertenece a una sola **categoría**.

```java
import jakarta.persistence.*;
import java.util.List;

@Entity
public class Categoria {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @OneToMany(mappedBy = "categoria", cascade = CascadeType.ALL)
    private List<Producto> productos;

    // Getters y setters
}
```



class Producto

```java
import jakarta.persistence.*;

@Entity
public class Producto {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @ManyToOne
    @JoinColumn(name = "categoria_id") // clave foránea
    private Categoria categoria;

    // Getters y setters
```



2. @OneToOne

Ejemplo: Un **usuario** tiene un solo **perfil**.

```java
@Entity
public class Usuario {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(name = "perfil_id", referencedColumnName = "id")
    private Perfil perfil;
}
```

1 Relacion de uno a uno

```java
@Entity
public class Perfil {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String descripcion;
}
```

3. @ManyToMany

Ejemplo: Un **estudiante** puede estar inscrito en muchos **cursos**, y un **curso** puede tener muchos **estudiantes**.

java

```java
@Entity
public class Estudiante {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @ManyToMany
    @JoinTable(
```



```
    @ManyToMany
    @JoinTable(
        name = "estudiante_curso",
        joinColumns = @JoinColumn(name = "estudiante_id"),
        inverseJoinColumns = @JoinColumn(name = "curso_id")
    )
    private List<Curso> cursos;
}
```



1 Relacion de muchos a muchos

```java
@Entity
public class Curso {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nombre;

    @ManyToMany(mappedBy = "cursos")
    private List<Estudiante> estudiantes;
}
```

🛠️ Consideraciones

- Usa `cascade = CascadeType.ALL` cuando quieras que las operaciones sobre una entidad también afecten a la entidad relacionada.
- Usa `fetch = FetchType.LAZY` o `EAGER` para definir cómo se cargan las relaciones.
- `mappedBy` se usa en el lado **no propietario** de la relación para evitar que JPA cree columnas duplicadas.



Descripcion

✅ `@JoinColumn`

Uso: Se utiliza para definir la **columna de clave foránea** (foreign key) en una relación `@ManyToOne` o `@OneToOne`.

Ejemplo:

```java
@ManyToOne
@JoinColumn(name = "categoria_id")
private Categoria categoria;
```

- `name`: nombre de la columna en la tabla actual (clave foránea).
- `referencedColumnName` (opcional): nombre de la columna en la tabla referenciada (por defecto es la columna `id`).

Resultado: En la tabla `producto`, se crea una columna `categoria_id` que referencia a `categoria.id`.

Descripcion

✅ `@JoinTable`

Uso: Se usa para definir una **tabla intermedia** en relaciones `@ManyToMany`, y también en algunos casos con `@OneToMany` bidireccionales.

Ejemplo:

```
@ManyToMany
@JoinTable(
    name = "estudiante_curso",
    joinColumns = @JoinColumn(name = "estudiante_id"),
    inverseJoinColumns = @JoinColumn(name = "curso_id")
)
private List<Curso> cursos;
```

Descripcion

Explicación de atributos:

- `name` : nombre de la tabla intermedia.
- `joinColumns` : define la clave foránea hacia **la entidad actual** (en este caso `Estudiante`).
- `inverseJoinColumns` : define la clave foránea hacia **la otra entidad** (en este caso `Curso`).

Resultado: Se crea una tabla `estudiante_curso` con dos columnas:

- `estudiante_id` (referencia a `Estudiante`)
- `curso_id` (referencia a `Curso`)

Práctica

4 Salidas

Cierre

- ¿Qué es una JavaScript?



Imagen obtenida de:
https://dpemfoco.com.br/wp-content/uploads/2019/08/Departamento-de-Pessoal-em-Foco-Checklist-Check-list-Checklists.png

Bibliografía

- Tanenbaum & Van Steen (2008). *Algoritmos y Estructuras de Datos - Principios y Paradigmas, 2da Edición*. Pearson Education.
- Khalid A. Mughal & Rolf W. Rasmussen (2017). A *Programmer's guide to Java SE 8 Oracle Certified Associate*.
- Schildt, Herbert (2019). *A Java Beginner's Guide Eighth Edition*. Oracle Press.
- Schildt, Herbert (2019). *Java The Complete Reference Eleventh Edition*. Oracle Press.



Imagen obtenida de:
https://pngimg.com/d/book_PNG51088.png



Universidad
Tecnológica
del Perú